Experiment 6: Object-Oriented Design – Use Case Model and Class Model

Tool used: StarUML 7.1.1
Diagram types: UML Use Case Diagram, UML Class Diagram (domain model and detection pipeline)
Subject system: Orbital Sentinel – Satellite Telemetry Anomaly Detection System

Field	Value
Student name	Pushkar Kumar Mishra
Reg. number	24BPS1128
Course	BCSE301P
Date of submission	22nd October, 2026
StarUML project file	docs/uml/Exp6_UseCase_Class.mdj
Source code	https://github.com/ItsPM25/Orbital_sentinel

1. Aim

To analyse the problem statement of the Orbital Sentinel system and design its object-oriented models: a Use Case Diagram that captures who uses the system and for what, and a Class Diagram that captures the classes, their attributes and operations, and the relationships between them.

2. Problem Statement

Satellites stream telemetry from thousands of sensor channels, far more than an operator can watch. Orbital Sentinel learns what normal telemetry looks like for each channel with a small LSTM network, replays NASA's expert-labelled SMAP/MSL telemetry through those models and flags readings that deviate from what the model expected. Each anomaly is attributed to the channel that caused it, filtered so that one-off spikes do not raise alarms (persistence filter), given a severity and a 1–10 risk level, and ranked across the fleet. Analysts see a risk-ranked alert list, a health panel with the telemetry chart, and a live 3D globe that shows five real satellites at their current positions computed from public orbital data (TLEs from CelesTrak). Administrators run the monitoring sessions and set thresholds; evaluators review accuracy metrics and the detection log.

System boundary: everything inside the Orbital Sentinel web application and its FastAPI backend. CelesTrak (the public TLE service) is outside the boundary and is modelled as a secondary actor. The system detects anomalies; it does not forecast failures, and health telemetry is a replay.

3. Identified Actors

Actor	Type	Description
User	Abstract primary actor	Any signed-in person. Holds the use cases every role can perform (log in, view alerts, inspect health, track the fleet).
Analyst	Primary actor (specialises User)	Operator who watches the alert list and acknowledges alerts.
Evaluator	Primary actor (specialises User)	Read-only reviewer of accuracy metrics, the detection log and the trained models.
Administrator	Primary actor (specialises Analyst and Evaluator)	Runs monitoring sessions, sets thresholds and imports the dataset; inherits everything an Analyst and an Evaluator can do.
CelesTrak	Secondary actor «external system»	Public GP API that supplies Two-Line Element sets (TLEs) for the satellites.

4. List of Use Cases

ID	Use case	Actor(s)	Description	SRS ref.
UC-01	Log in	User	Sign in with e-mail and password; a signed httpOnly cookie carries the role.	SRS 5.3
UC-02	View risk-ranked alerts	User	Alert list ordered by risk with verdict, severity and attributed channel.	REQ-16
UC-03	Inspect satellite health	User	Health panel: verdict, risk, anomaly score, attributed channels.	REQ-15
UC-04	View telemetry chart	(extends UC-03)	Telemetry plot with flagged ranges shaded; channel selector.	REQ-17
UC-05	Track fleet on 3D globe	User	Real positions from TLE + SGP4, coloured by risk.	REQ-12–14
UC-06	Change simulation speed	(extends UC-05)	Live / 10× / 60× / 300× globe clock.	REQ-13
UC-07	Fetch TLE data	CelesTrak (included by UC-05)	Download TLEs, cache them for 2 h, fall back to a bundled snapshot.	REQ-12
UC-08	Acknowledge alert	Analyst	Moves an alert from AlertRaised to Acknowledged (Exp 5 state machine).	REQ-16
UC-09	View accuracy metrics	Evaluator	Precision, recall, F1 and FPR, with and without the persistence filter.	REQ-5, REQ-11
UC-10	Browse detection log	Evaluator	Every detection with timestamp, score, channel, severity and outcome.	REQ-18
UC-11	View trained models	Evaluator	Model registry (MDL-LSTM-100-80, 81 channels).	REQ-3
UC-12	Start/stop monitoring session	Administrator	Starts the per-satellite replay → LSTM → lifecycle pipeline.	REQ-1–4
UC-13	Configure thresholds	Administrator	Persistence N and severity threshold per satellite.	REQ-8, REQ-9
UC-14	Import dataset	Administrator	Register the NASA SMAP/MSL channels against the satellites.	REQ-1

5. Use Case Relationships

Relationship	From	To	Meaning
Association	Actors	Their use cases	The actor initiates the use case (14 associations).
«include»	Track fleet on 3D globe	Fetch TLE data	The globe always needs current TLEs.
«extend»	Inspect satellite health	View risk-ranked alerts	Optional drill-down from an alert row.
«extend»	Inspect satellite health	Track fleet on 3D globe	Optional: clicking a satellite on the globe opens its panel.
«extend»	View telemetry chart	Inspect satellite health	Optional: open the chart from the health panel.
«extend»	Change simulation speed	Track fleet on 3D globe	Optional change of the globe clock.
Generalization	Analyst, Evaluator	User	Both roles inherit the common signed-in use cases.
Generalization	Administrator	Analyst, Evaluator	The Administrator can do everything the other two roles can.

6. Use Case Diagram

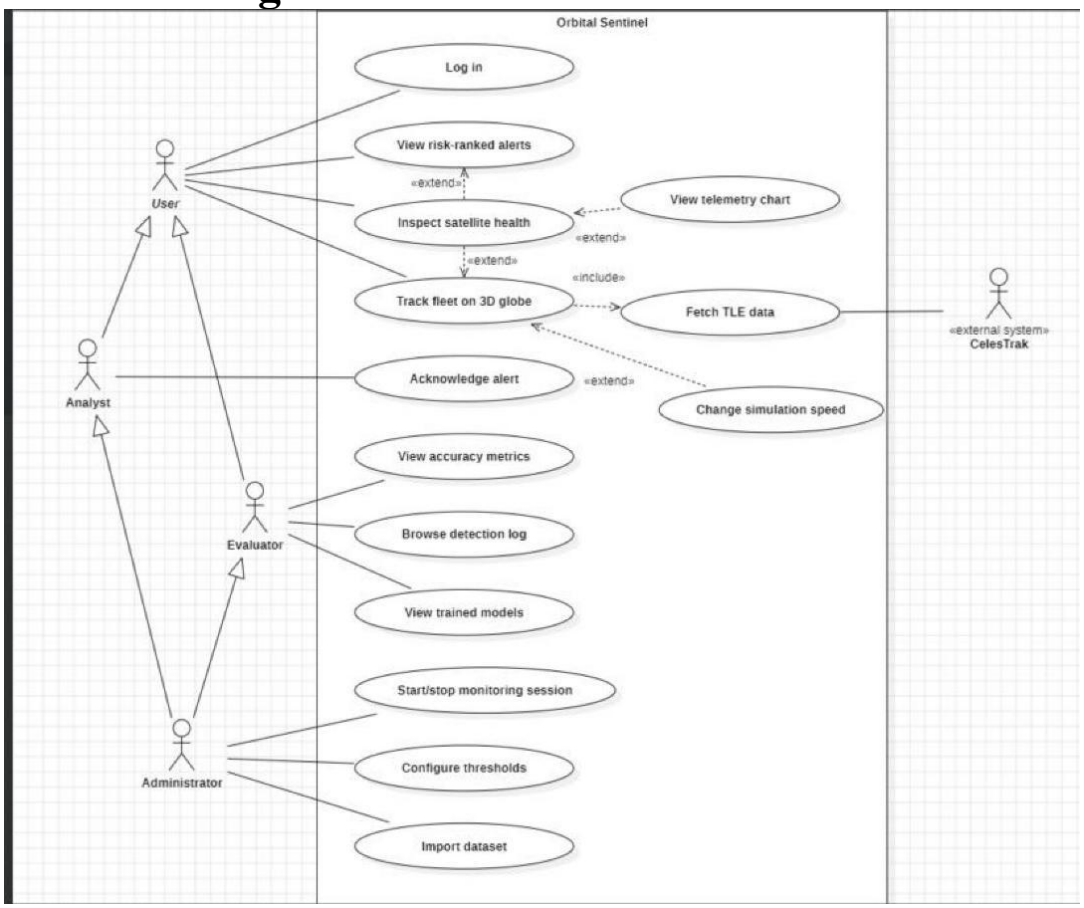


Figure 1: Use Case Diagram – Orbital Sentinel

7. Identified Classes

Nouns in the problem statement and the ER diagram (Exp 4) became the domain (entity) classes; the processing steps became the pipeline (control) classes. Every class maps to real code.

Class	Kind	Responsibility	Code
Satellite	Entity	A monitored spacecraft with its TLE and proxy flag.	models/tables.py
Channel	Entity	One telemetry sensor channel of a satellite.	models/tables.py
TelemetryReading	Entity	One timestamped value of a channel.	models/tables.py
DetectionModel	Entity	A trained detector (weights path, hyper-parameters).	models/tables.py
Detection	Entity	One scored anomalous window (score, severity, outcome).	models/tables.py
Attribution	Entity	Share of a detection blamed on one channel (top 3).	models/tables.py
Alert	Entity	An alert raised from a confirmed detection; carries the alert state.	models/tables.py
User	Entity	A person with one Role and an argon2 password hash.	models/tables.py
MonitoringConfig	Entity	Persistence N and severity threshold set for a satellite.	models/tables.py
Role, AlertStatus, SeverityLevel, DetectionOutcome	Enumerations	Fixed value sets; the names match the SRS and the Exp 5 state diagram.	models/tables.py
SessionRegistry	Control	Starts, stops and lists running monitoring sessions.	services/sessions.py
MonitoringSession	Control	Runs replay → score → lifecycle → store → publish for one satellite.	pipeline/session.py

ReplaySource, Normaliser, Windower	Control	Replay the dataset as a time-ordered stream of windows.	pipeline/ingest.py
Detector «interface»	Interface	score(window) → ChannelScores; lets stages be swapped and mocked.	pipeline/detect.py
LstmDetector, ScriptedDetector	Control	Real LSTM detector, and the test double used by pytest.	pipeline/detect.py
LstmPredictor	Control	PyTorch network: 2 LSTM layers × 80 units + linear head.	pipeline/lstm.py
StreamingThreshold	Control	Non-parametric dynamic threshold over the last 2,100 errors.	pipeline/detect.py
DetectionLifecycle	Control	The Exp 5 state machine (persistence filter, Confirmed/Suppressed).	pipeline/lifecycle.py
TelemetryWindow, ChannelScores	Data (value objects)	Data passed between the pipeline stages.	pipeline/types.py
Explain «utility»	Utility	Attribution, severity, risk label, fleet ranking.	pipeline/explain/
EventBus	Control	In-process publish/subscribe for live (SSE) updates.	services/events.py
CelesTrakClient «utility»	Boundary	Fetches and caches TLEs from the external CelesTrak API.	tracking/celestrak.py

8. Class Diagram

The class model is split into two diagrams in the same StarUML project so each stays readable: the domain model (persistent entities) and the detection pipeline (behaviour). The Satellite class appears on both diagrams as the same model element.

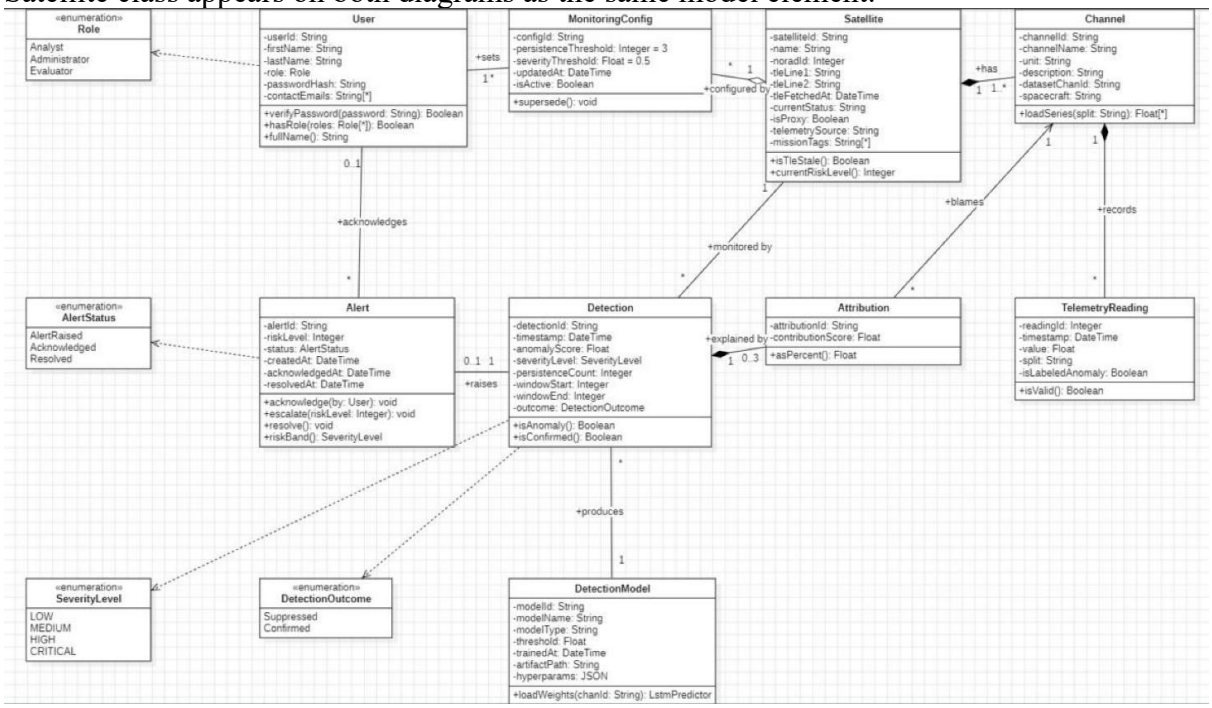


Figure 2: Class Diagram – Domain Model

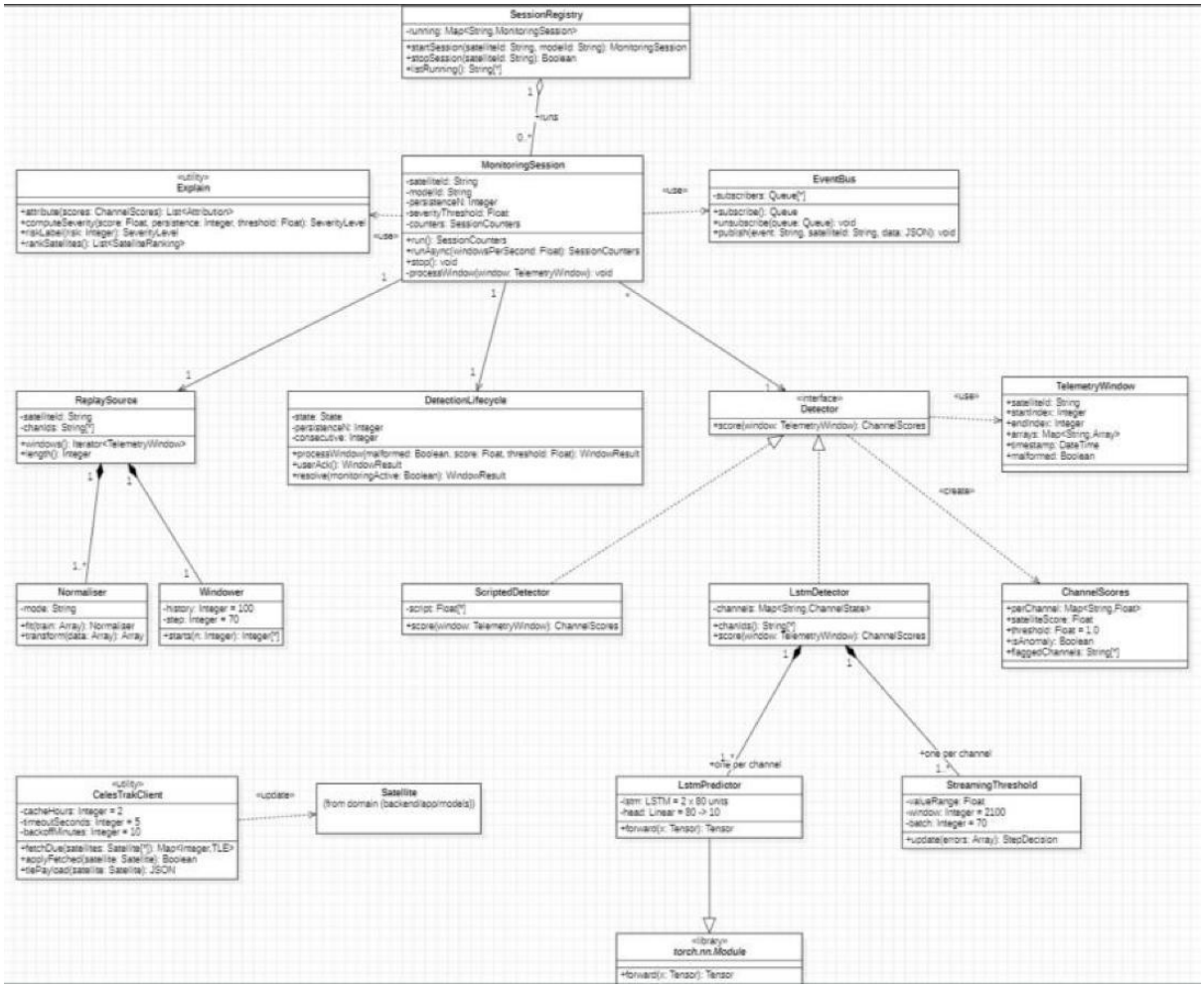


Figure 3: Class Diagram – Detection Pipeline

9. Relationships in the Class Model

From	To	Type	Multiplicity	Meaning
Satellite	Channel	Composition	1 → 1..*	A channel cannot exist without its satellite.
Channel	TelemetryReading	Composition	1 → *	Readings belong to one channel.
Detection	Attribution	Composition	1 → 0..3	Up to three blamed channels per detection.
Satellite	MonitoringConfig	Aggregation	1 → *	Config history is kept; only the newest row is active.
Satellite	Detection	Association	1 → *	Detections are made for a satellite.
DetectionModel	Detection	Association	1 → *	Each detection records the model that produced it.
Attribution	Channel	Association (directed)	* → 1	Each attribution blames one channel.
Detection	Alert	Association	1 → 0..1	A confirmed detection raises at most one alert.
User	Alert	Association	0..1 → *	The user who acknowledged the alert.
User	MonitoringConfig	Association	1 → *	The administrator who set the config.
SessionRegistry	MonitoringSession	Aggregation	1 → 0..*	The registry holds the running sessions.
ReplaySource	Normaliser, Windower	Composition	1 → 1..* / 1	Ingestion helpers owned by the source.
LstmDetector	LstmPredictor, StreamingThreshold	Composition	1 → 1..*	One predictor and one threshold per channel.

LstmDetector, ScriptedDetector	Detector	Realization	–	Both implement the Detector interface.
LstmPredictor	torch.nn.Module	Inheritance (generalization)	–	The network is a PyTorch module.
MonitoringSession	Explain, EventBus	Dependency «use»	–	Called per window; not stored as fields.
Detector	TelemetryWindow / ChannelScores	Dependency «use» / «create»	–	Input and output of score().
CelesTrakClient	Satellite	Dependency «update»	–	Writes fetched TLEs to the satellite row.

10. Brief Explanation of Design Decisions

Abstract User actor. All three roles share the signed-in use cases, so they sit on an abstract User actor. Analyst and Evaluator specialise User, and Administrator specialises both, which matches the server-side role checks (require_roles) without drawing duplicate association lines.

Log in as its own use case. Authentication is shown once, on User, instead of 13 «include» arrows from every use case; the rule that every request needs a session is enforced by the server on each call.

Include vs extend. Fetching TLEs is mandatory for the globe, so it is «include». Opening the health panel, the telemetry chart and changing the clock speed are optional user choices, so they are «extend».

Names traced to earlier experiments. Entity names are the ER diagram's (Exp 4) and the alert states are the state diagram's (Exp 5), so the design can be followed straight into the code.

Encapsulation. Entity attributes are private (–) and are changed only through operations such as Alert.acknowledge() and Alert.escalate(), which enforce the AlertRaised → Acknowledged → Resolved order.

Abstraction through an interface. MonitoringSession depends on the Detector interface, not on the LSTM. ScriptedDetector realises the same interface, so the whole pipeline is unit-tested without a trained model.

Composition only where lifetimes are bound. Channels, readings and attributions are destroyed with their owner, so they are compositions; monitoring configs keep their history, so the satellite only aggregates them.

Multivalued attributes. missionTags and contactEmails are shown as String[*] attributes; the database stores them in separate tables only to satisfy first normal form, which is an implementation detail.

No duplicate classes. Stateless helper functions (attribution, severity, ranking, CelesTrak access) are grouped into «utility» classes rather than invented as separate objects.

11. Full-Screen Screenshots (with timestamp)

Screenshots of StarUML with the project open; the Windows taskbar clock shows the date and time.

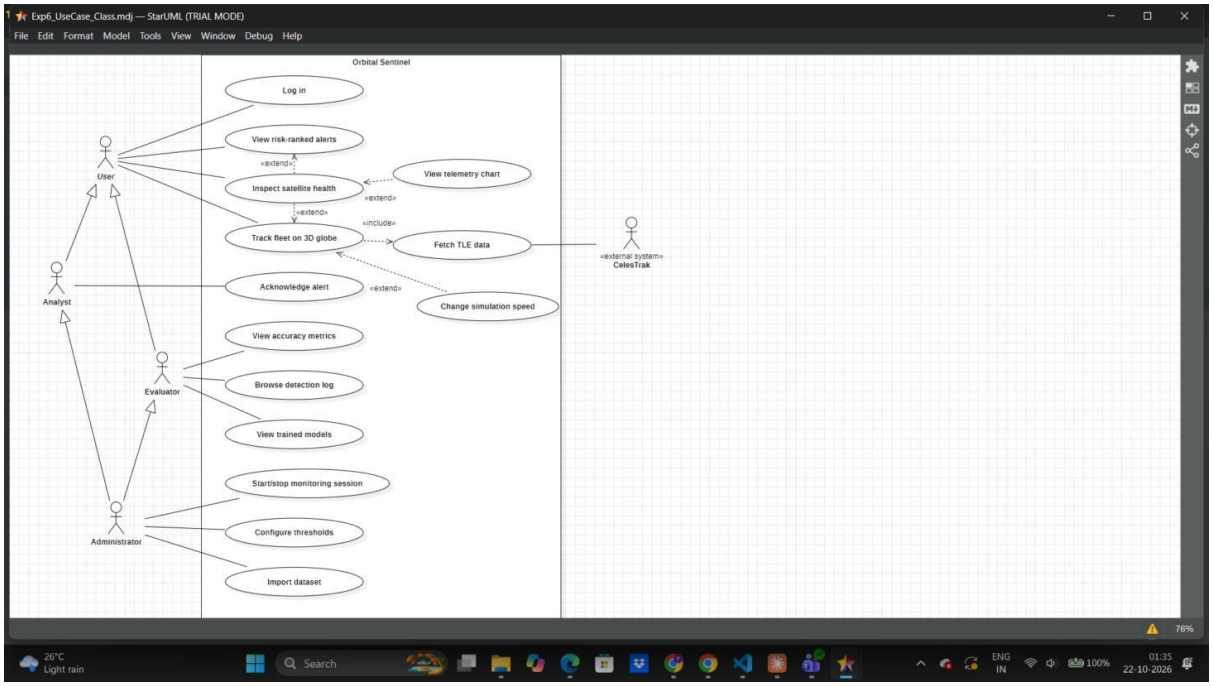


Figure 4: Use Case Diagram open in StarUML, with system timestamp

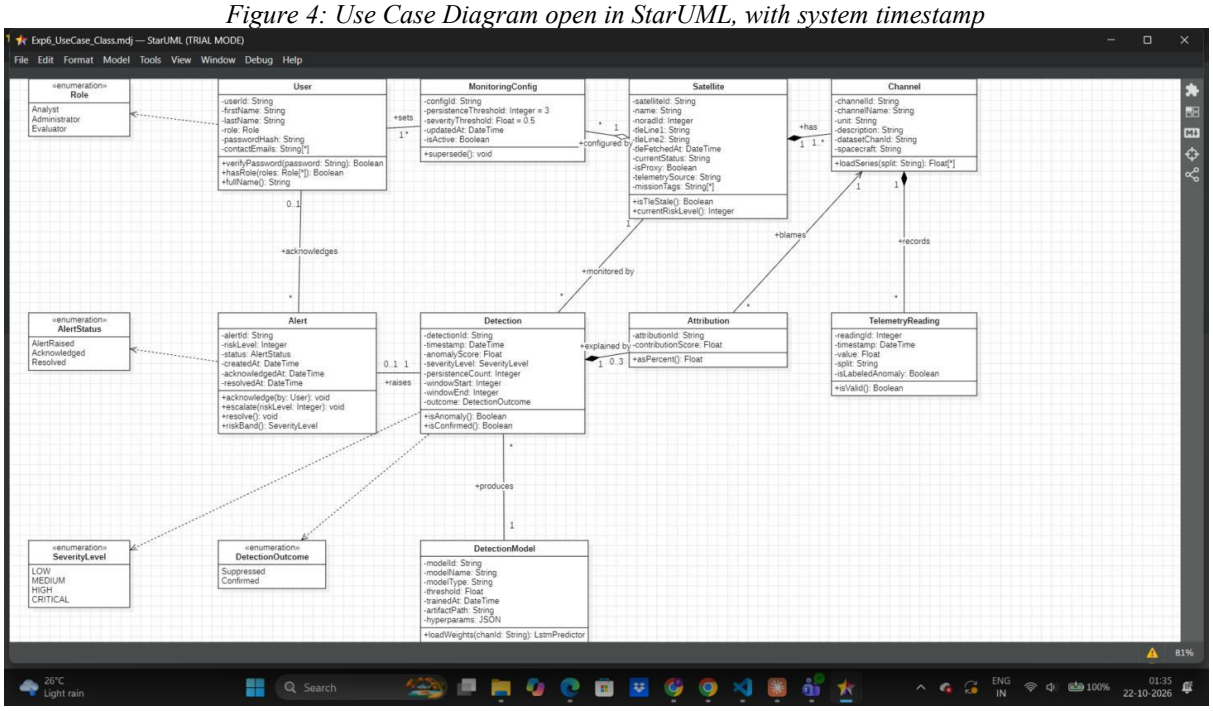


Figure 5: Domain Class Diagram open in StarUML, with system timestamp

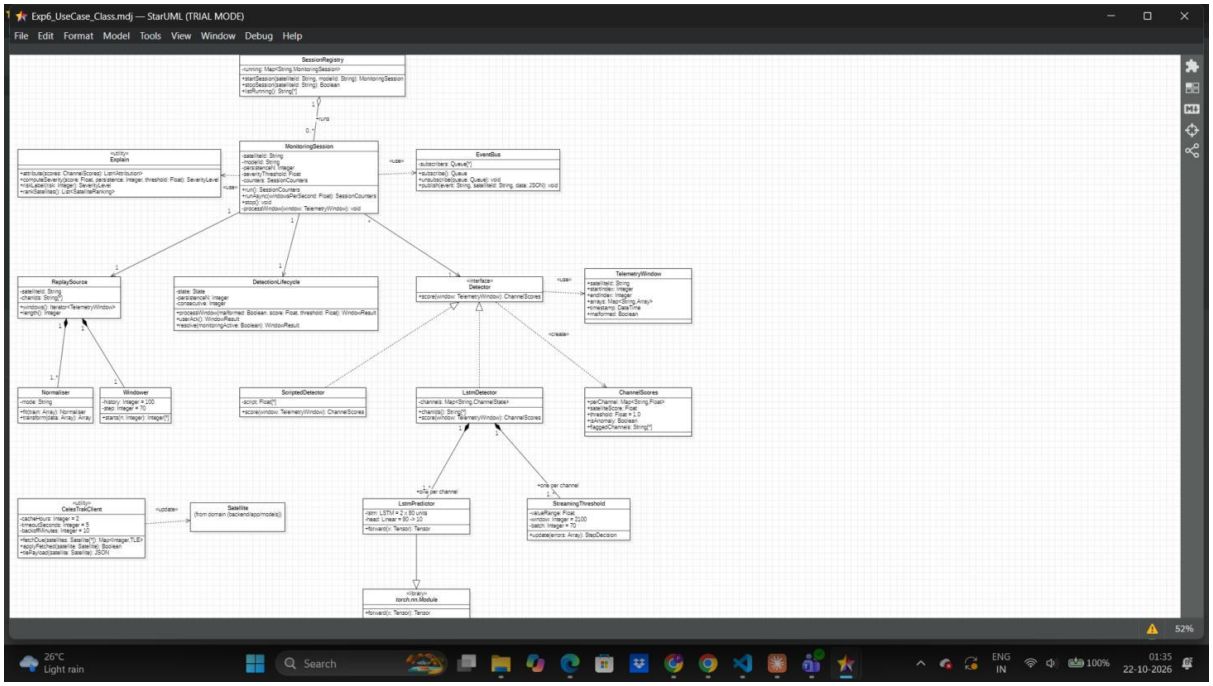


Figure 6: Detection Pipeline Class Diagram open in StarUML, with system timestamp

12. Submission Checklist

☑ Problem statement analysed and system boundary identified (Section 2)

☑ Actors identified, including a secondary external system (Section 3)

☑ Use cases listed and traced to the SRS requirements (Section 4)

☑ Use Case Diagram with association, include, extend and generalization (Figure 1)

☑ Classes identified with attributes and operations (Section 7)

☑ Class Diagram with association, aggregation, composition, inheritance, realization and dependency (Figures 2, 3)

☑ Design decisions explained (Section 10)

☑ Timestamp screenshots attached (Section 11)

☑ StarUML project file saved (docs/uml/Exp6_UseCase_Class.mdj)

13. Result

The Use Case Model and the Class Model of the Orbital Sentinel system were designed in StarUML using standard UML notation. The use case diagram shows 5 actors and 14 use cases with association, include, extend and generalization relationships. The class model shows 9 entity classes, 4 enumerations and 16 pipeline classes (plus the torch.nn.Module library class) with their attributes, operations and all required relationship types. Every class name matches the implementation in the project repository.